Table des matières

Exercices corrigés 09 — Rebâtir le projet de bout en bout 1
Questions courtes 1
Questions pratiques 3
Diagnostics 3
Questions à développer 4
Mises en situation 4
Corrigé 6

Exercices corrigés 09 — Rebâtir le projet de bout en bout

Comment travailler ces exercices. Répondez seul, puis vérifiez avec le corrigé. Les exercices marqués [labo] portent sur ce que vous venez de faire au labo ; les autres vérifient que le cours est clair dans votre tête. Tout vient de la Théorie 09 et du Labo 09, et des huit cours qu'ils rassemblent.

Les repères. Les commandes dbt se lancent depuis `Projet_Boreal_intermediaire/02_ae`, les requêtes DuckDB depuis `Projet_Boreal_intermediaire`, sur l'entrepôt refait au labo. Un essai qui abîme l'entrepôt se fait sur une copie : `copy plateforme\entrepot\boreal.duckdb` vers un autre dossier, avec une cible de plus dans votre `profiles.yml`. Un essai qui touche au code se défait par `git restore`.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Les trois promesses. Nommez les trois promesses d'un projet fini, et dites laquelle est la seule à trouver des défauts.
2. De zéro. Dites ce que la reconstruction d'Analytics Engineering efface, ce qu'elle garde, et pourquoi elle garde ce qu'elle garde.
3. [labo] La copie neuve. Nommez trois choses que votre dossier de travail avait et que la copie de l'étape 6 n'avait pas, et dites pourquoi la reconstruction part de la copie.
4. [labo] Ce que le dépôt suit. Donnez le nombre de fichiers suivis à la fin du labo, et nommez trois choses du projet qui n'y sont pas, chacune avec sa raison.
5. Un état, pas une histoire. Dites pourquoi l'histoire des paliers se refait chez Boréal, et à quelle condition elle se referait ailleurs.

© 2026 iAlexMG. Tous droits réservés.

6. [labo] Sans les extraits. Donnez ce que rend une reconstruction qui ne rejoue que l'état courant du référentiel, et dites pourquoi aucun test ne tombe.
7. Sauvegarder ou refaire. Dites quelle partie du projet se sauvegarde plutôt que de se refaire, quand la source ne garde rien, et ce que dbt en fait par défaut sur Snowflake d'après sa documentation.
8. L'ordre. Donnez les neuf commandes de `reconstruire.cmd` dans l'ordre, et la raison de trois d'entre elles.
9. La cinquième passe. Dites ce qu'elle capture, pourquoi elle n'a rien ajouté sur Boréal, et ce qu'elle aurait fait sur une génération qui s'arrête en décembre 2025.
10. `|| exit /b 1`. Dites ce que cette fin de ligne fait, et ce que le script annoncerait sans elle après un échec.
11. [labo] La garde. Dites ce qu'elle compte, ce qu'elle a répondu sur votre entrepôt avant l'étape 5, et pourquoi elle n'efface rien elle-même.
12. [labo] Par-dessus. Donnez ce qu'une reconstruction lancée sans garde sur un entrepôt construit fait au snapshot, et ce que les tests en voient.
13. La table orpheline. Définissez-la, dites pourquoi `dbt build` ne la voit pas, et donnez les deux outils du capstone qui la voient.
14. [labo] Le temps. Dites où passent les 68 secondes de la reconstruction, et la leçon qu'on en tire.
15. Mettre de côté. Dites pourquoi on copie l'entrepôt avant d'effacer, et quelle preuve on perd sans cette copie.
16. Refaire n'est pas prouver. Dites ce que `Reconstruction : au vert` garantit, et ce qu'il ne garantit pas.
17. `except all`, dans les deux sens. Dites pourquoi `all`, et pourquoi les deux sens.

1. [labo] Les vues. Dites pourquoi le script compare une vue par sa définition, et donnez la mesure qui le justifie.
2. Les témoins. Définissez un témoin indépendant, et nommez les cinq du projet avec ce que chacun compare.
3. Ce que les témoins ne voient pas. Donnez un défaut qu'aucun des cinq ne voit, et la preuve qui le voit.
4. [labo] L'après-vente. Donnez les deux semaines d'assemblage qui ressortent, et leurs chiffres.
5. [labo] L'usine. Dites comment la bande habituelle de la vibration est fixée, ses deux valeurs, et les jours qui en sortent.
6. Joindre les deux bouts. Nommez les trois choix de conception, faits bien avant le capstone, qui permettent de croiser l'usine et l'après-vente.
7. Le `README.md`. Dites ce qu'un bon `README.md` de dépôt de données dit, ce qu'il ne dit pas, et comment on le met à l'épreuve.
8. L'étiquette. Dites ce qu'est une étiquette annotée, pourquoi on la pose, comment on la pousse, et ce qui ferait la `v2.0`.

© 2026 iAlexMG. Tous droits réservés.

Questions pratiques

P1. [labo] Deux environnements, une comparaison. Construisez le projet sur la cible `dev`, puis écrivez, dans la ligne de commande DuckDB, la requête qui compare `marts.fct_ventes` à `dev_marts.fct_ventes` dans les deux sens. Donnez le résultat, dites pourquoi on n'a pas besoin d'attacher un second fichier, et dites ce que la dimension des concessionnaires de `dev` lit. Jetez ensuite les schémas de travail.

P2. La garde qui regarde la livraison. Une reconstruction lancée sur un entrepôt où le schéma `argent` manque passe la garde, rejoue ses cinq passes du snapshot, puis échoue. Mesurez sur une copie à quel moment et sur quoi, puis étendez `exiger_entrepot_neuf` pour qu'elle refuse d'abord une livraison incomplète de Data Engineering : 22 tables dans `raw`, 3 dans `argent`.

P3. Le rapport d'exécution. Écrivez le script Python qui lit `target/run_results.json` après un `dbt build` et donne le nombre de résultats par type et par statut, puis les cinq nœuds les plus longs. Lancez-le deux fois de suite, et dites ce qui est stable et ce qui ne l'est pas.

P4. L'incident en deux lignes. Écrivez la requête qui range les jours d'assemblage de la ligne des motoneiges selon que la vibration du compresseur 2 était dans sa bande ou au-dessus, et donne pour chaque groupe le nombre de jours, de motoneiges assemblées, de réclamations de refroidissement et le taux.

P5. La liste du rappel. La cohorte du Cours 07, les motoneiges assemblées du 15 au 28 octobre 2025 sur la ligne `LMN`, doit être rappelée. Écrivez la requête qui compte les motoneiges vendues de la cohorte qui n'ont encore aucune réclamation de refroidissement, par province du concessionnaire qui les a vendues, et donnez le total et le nombre de concessionnaires à prévenir.

P6. La documentation sous surveillance. Écrivez `outils/verifier_descriptions.py`, qui rend le code 1 et nomme chaque modèle, seed, snapshot ou source sans description, puis l'étape du workflow qui le lance. Dites où l'étape se place, et vérifiez-la dans les deux sens.

P7. (avancé) Un test pour l'histoire. Écrivez un test singulier sur `snp_concessionnaire` qui signale une version qui finit avant de commencer et deux versions du même concessionnaire qui se chevauchent. Vérifiez-le sur l'entrepôt refait, puis sur une copie où la reconstruction a été lancée par-dessus le projet existant.

P8. (avancé) Le troisième bout. La télémétrie est un troisième chemin vers l'incident. Pour les motoneiges de la cohorte, comparez la date du premier code du circuit de refroidissement levé en roulant, `T310` ou `T320`, à celle de la première réclamation de refroidissement. Donnez le nombre de véhicules connectés, ceux qui ont levé un code, ceux qui ont les deux, l'écart médian, et ce qu'un responsable de la garantie ferait des autres.

Diagnostics

E1. Au vert, dans le vide. Un collègue retape les commandes de la reconstruction à la main, sans `--target prod`. Tout passe au vert, `PASS=128`. La comparaison rend `29 ecart(s)` et le code 1. Dites ce qui s'est passé, où sont les objets, et pourquoi un seul a atterri au bon endroit.

E2. Le changement oublié. Une collègue a corrigé dans `stg_regions` les deux libellés de province tronqués à vingt caractères, a reconstruit la production, tout au vert, et n'a pas commité. Vous lancez le capstone. La comparaison rend 1 326 lignes manquantes et 1 326 en trop dans `dim_client`, 13 et 13 dans `dim_concessionnaire`, et `stg_regions` à la définition différente : 2 679 écarts. Dites ce qui s'est passé, lequel des deux entrepôts a raison, et ce qu'on fait.

© 2026 iAlexMG. Tous droits réservés.

E3. Toutes les vues différentes. Un collègue refait le projet dans un fichier neuf, `boreal_neuf.duckdb`, pour garder l'ancien intact, et le compare à `boreal.duckdb`. Les seize tables sont égales, les quatorze vues sortent toutes `DIFFERENTE`. Dites pourquoi, et ce qu'il doit faire.

E4. La preuve qui se compare à elle-même. Une collègue trouve le script trop compliqué et compare toutes les relations par leurs lignes, vues comprises : 0 écart partout. Sur une copie amputée de 17 741 lignes de ventes, sa comparaison de `rpt_ventes_par_palier` rend toujours 0. Dites pourquoi, et ce que sa preuve vaut.

E5. Les lots dans le désordre. Un collègue réécrit `reconstruire.cmd` et intervertit deux lots : 2022-01, 2024-07, 2024-06, 2025-12. La garde passe, les cinq snapshots aussi, puis `PASS=122 ERROR=4 SKIP=3` : 645 lignes de vente en double, 164 enregistrements, 43 réclamations, la réconciliation sur 213 commandes. Les tests de `dim_concessionnaire` passent. Dites ce qui s'est passé, pourquoi ce sont les faits qui tombent, et ce qui l'aurait vu à la source.

Questions à développer

A. Refaire ou restaurer. Une équipe sauvegarde son entrepôt chaque nuit et n'a jamais essayé de le refaire depuis son dépôt. Une autre refait tout chaque dimanche et ne sauvegarde rien. Défendez ce que chacune fait bien, dites ce que chacune risque, et proposez ce qu'il faudrait à un projet comme celui de Boréal.

B. Fini. « Tous les tests passent, donc le projet est fini. » Critiquez cette phrase en vous appuyant sur ce que le capstone a mesuré.

C. (avancé) Une CI qui refait tout. On vous propose de remplacer la CI du Cours 08 par une reconstruction complète sur une copie des vraies données, à chaque demande de tirage, avec les trois preuves. Défendez ou critiquez, en chiffrant ce que vous pouvez.

Mises en situation

M1. Coopérative agricole de Roberval. Le projet dbt de la coopérative historise la catégorie de ses membres par un snapshot, sur un logiciel de gestion qui ne livre que l'état courant. L'équipe veut « tout refaire de zéro chaque mois, pour être propre ». Dites ce que vous acceptez, ce que vous refusez, et ce que vous mettez en place.

M2. (avancé) Distributeur alimentaire de Rivière-du-Loup. Un projet de 140 modèles et 30 snapshots se reconstruit en 25 minutes, dont plus de 20 pour les snapshots, chacun rejoué sur 24 lots mensuels. Un audit trouve 11 tables dans `marts` qu'aucun modèle ne fabrique. On vous demande une reconstruction plus courte et des preuves. Proposez un plan, dans l'ordre.

M3. Pêcheries de Gaspé. Vous quittez l'entreprise dans trois semaines, et un analyste arrivé la semaine dernière reprendra votre projet dbt. Rien n'est écrit, hors du code. Dites ce que vous écrivez, dans quel ordre, et comment vous saurez que c'est suffisant avant de partir.

© 2026 iAlexMG. Tous droits réservés.

© 2026 iAlexMG. Tous droits réservés.

Corrigé

Comment lire ce corrigé. Il donne la réponse et le pourquoi, jamais une notion neuve : tout ce qu'il emploie a été enseigné dans les neuf cours de la formation. Les chiffres sortent d'exécutions réelles sur le projet de Boréal tel que le Labo 09 le laisse, et sur des copies de son entrepôt abîmées exprès, jamais sur l'entrepôt de travail.

Questions courtes

1. Les trois promesses. Il se refait : le dépôt et ce que Data Engineering livre suffisent. Il se prouve : ce qui revient égale ce qui était là, et l'étoile répond encore à ses questions. Il se passe : une autre personne le reprend sans vous. La première est la seule qui trouve des défauts, parce qu'elle rejoue par écrit tous les gestes faits une fois à la main, et c'est là que les oublis se montrent. *Théorie : `Theorie_09_AE.md`, section 1.*

2. De zéro. Elle efface les quatre schémas qu'Analytics Engineering écrit, `staging`, `marts`, `presentation` et `snapshots`. Elle garde `raw`, `argent` et `pipeline`, qui appartiennent à Data Engineering et que le capstone de Data Engineering refait de son côté, et le profil de connexion, qui n'a jamais été dans le dépôt. On ne refait que ce qu'on produit. *Théorie : `Theorie_09_AE.md`, section 1.*

3. La copie neuve. `dbt_packages/`, installé au Cours 04 ; `target/`, la documentation et le manifeste ; `logs/`. Tout ce que le `.gitignore` écarte, et tout fichier modifié mais pas commité. La reconstruction part de la copie parce que la copie est ce qu'un collègue, un runner de CI ou un kit recevront : ce que Git suit, rien de plus. Un projet qui ne se refait que dans votre dossier dépend de votre dossier. *Théorie : `Theorie_09_AE.md`, section 1.*

4. Ce que le dépôt suit. 63 fichiers. Pas dedans : le profil de connexion, parce qu'il porterait un chemin du poste et, sur une plateforme, un secret ; `dbt_packages/`, qui se réinstalle par `dbt deps` aux versions de `package-lock.yml` ; les données de l'entrepôt, qui se refont par `dbt build` ou viennent de Data Engineering. On aurait pu nommer aussi `target/` et `logs/`, qui se régénèrent à chaque commande. *Théorie : `Theorie_09_AE.md`, section 3.*

5. Un état, pas une histoire. Les tables se dérivent de la `RAW` telle qu'elle est ; l'histoire des paliers se dérive de ce qu'était le référentiel chaque mois. Elle se refait chez Boréal parce que Data Engineering garde chaque extrait mensuel du référentiel, 52 lots à l'état de mars 2026. Ailleurs, elle se referait à la même condition : une source qui garde ses extraits datés. Une source qui ne livre que son état courant ne permet de refaire que cet état. *Théorie : `Theorie_09_AE.md`, section 3.*

6. Sans les extraits. `PASS=128`, tous les tests au vert, 156 versions au lieu de 205, aucune fermée. 17 733 lignes de vente changent de palier, pour 94 628 792,88 $, et le total reste 465 924 613,14 $ au cent près. Aucun test ne tombe parce qu'aucun ne dit à quel palier une vente appartient : la réconciliation compare des totaux, le grain compte des lignes, le test unitaire travaille sur ses propres données. C'est le type 1 du Cours 04, revenu par une reconstruction. *Théorie : `Theorie_09_AE.md`, section 3.*

7. Sauvegarder ou refaire. Le schéma `snapshots`, la seule partie du projet qui accumule ce que la source ne garde pas. D'après la documentation de dbt, consultée le 2026-09-23, dbt crée toutes ses tables Snowflake en *transient* par défaut, sans protection de récupération après suppression et avec au plus un jour de voyage dans le temps, et la page ne fait pas d'exception pour le snapshot. Une histoire qu'aucune source ne peut refaire se déclare `transient: false`, et se sauvegarde. *Théorie : `Theorie_09_AE.md`, section 3.*

© 2026 iAlexMG. Tous droits réservés.

8. L'ordre. `dbt deps` ; `dbt run-operation exiger_entrepot_neuf` ; quatre `dbt snapshot` aux lots 2022-01, 2024-06, 2024-07 et 2025-12 ; `dbt snapshot` sans lot ; `dbt build` sans le snapshot ; `dbt docs generate`. Trois raisons au choix : `deps` d'abord, parce que rien ne compile sans `dbt_utils`, pas même la garde ; la garde avant tout ce qui écrit, pendant qu'il n'y a rien à abîmer ; les lots dans l'ordre, parce que la stratégie `check` compare des valeurs à ce qu'elle a enregistré, pas des dates ; le snapshot avant `dbt build`, que `dim_concessionnaire` lit ; la documentation en dernier, puisque le catalogue lit les objets construits. *Théorie : `Theorie_09_AE.md`, section 4.*

9. La cinquième passe. Elle capture l'état courant, le dernier lot livré, le geste qu'un ordonnanceur ferait chaque nuit. Sur Boréal, aucun palier ne change dans les trois lots de 2026, donc elle n'ouvre aucune version : 205 avant, 205 après. Sur une génération qui s'arrête en décembre 2025, le dernier lot est 2025-12, déjà rejoué : la passe ne change rien non plus, et c'est pour cela qu'elle peut vivre dans le script sans condition. *Théorie : `Theorie_09_AE.md`, section 4.*

10. `|| exit /b 1`. Il arrête le script dès qu'une commande rend un code non nul, et fait rendre le code 1 au script. Sans lui, le script passerait à la commande suivante, et la dernière ligne afficherait `Reconstruction : au vert` après un snapshot ou une construction ratés. Un vert qui ne dépend de rien est pire que pas de message du tout. *Théorie : `Theorie_09_AE.md`, section 4.*

11. La garde. Les objets des quatre schémas du projet, dans l'`information_schema`. Sur l'entrepôt construit : `marts` 15 objets, `presentation` 1, `snapshots` 1, `staging` 13, puis l'erreur et le code 1. Elle n'efface rien parce qu'effacer est une décision : un script qui efface ce qu'il va refaire effacerait un jour le mauvais entrepôt, et une garde qui refuse oblige quelqu'un à regarder. *Théorie : `Theorie_09_AE.md`, section 4.*

12. Par-dessus. Le snapshot passe de 205 versions à 303, dont 49 qui finissent avant de commencer : le lot de 2022, présenté après l'état de 2025, ferme des versions en arrière. Les tests en voient une partie : deux tests de `dim_concessionnaire` tombent, le grain sur 49 combinaisons et l'unicité de la clé sur 98, et 41 nœuds sont sautés. Ici, les tests le disent ; mais le snapshot, lui, est déjà abîmé, et sur une source qui ne garde pas ses extraits, rien ne le répare. *Théorie : `Theorie_09_AE.md`, section 4.*

13. La table orpheline. Une table qu'aucun modèle du dépôt ne fabrique plus, restée dans l'entrepôt après un renommage ou une suppression. `dbt build` ne construit que ce que le projet déclare et ne jette jamais ce qu'il ne connaît pas : mesuré, une table de 19 636 lignes ajoutée à `marts` survit à la construction, et `dbt ls` ne la nomme pas. La garde la voit, puisqu'elle compte tous les objets des schémas du projet ; la comparaison aussi, qui la signale « présente d'un seul côté ». *Théorie : `Theorie_09_AE.md`, section 4.*

14. Le temps. 68 secondes : une dizaine pour construire 129 nœuds, quarante pour les cinq passes du snapshot, le reste pour les paquets, la garde et la documentation. Chaque passe du snapshot dure huit secondes, dont moins d'une pour traiter 156 lignes : le démarrage de dbt, qui relit le projet à chaque commande, coûte plus que le travail. On compte les commandes, pas les lignes. *Théorie : `Theorie_09_AE.md`, section 4.*

15. Mettre de côté. Parce que la première preuve compare le refait à ce qui était là, et que ce qui était là n'existe plus une fois effacé. Sans la copie, on perd la preuve par les lignes, et il ne reste que les témoins et l'incident : un défaut qu'aucun témoin ne couvre, comme une vente attribuée au mauvais palier, passerait. *Théorie : `Theorie_09_AE.md`, section 3.*

16. Refaire n'est pas prouver. Il garantit que chaque commande a rendu 0 : la compilation, les constructions et les 98 tests de données et le test unitaire. Il ne garantit pas que le refait égale l'avant, ni qu'il dit vrai sur ce qu'aucun test ne couvre. Le Cours 05 l'a montré : un snapshot daté de l'heure du rejeu faussait les ventes par palier, tous les tests au vert. *Théorie : `Theorie_09_AE.md`, section 5.*

© 2026 iAlexMG. Tous droits réservés.

17. `except all`, dans les deux sens. `all` compte les exemplaires : sans lui, une ligne présente deux fois d'un côté et une fois de l'autre disparaîtrait de la différence, et un doublon passerait. Les deux sens parce que chacun voit un défaut différent : ce qui manque au refait, et ce qu'il porte en trop. Un seul sens laisse passer la moitié des erreurs. *Théorie : `Theorie_09_AE.md`, section 5.*

18. Les vues. DuckDB range dans chaque vue le nom de la base qu'elle lit, `boreal.marts.fct_ventes`, et les deux fichiers s'appellent `boreal` : attachée à côté du refait, une vue de l'entrepôt d'avant lit les tables du refait. Mesuré : 17 741 lignes retirées exprès de l'ancien fait des ventes, que la comparaison des tables voit toutes, pendant que la vue de présentation de l'ancien entrepôt compte toujours 80 618 lignes. On compare donc la définition ; le contenu suit des tables, comparées. *Théorie : `Theorie_09_AE.md`, section 5.*

19. Les témoins. Un témoin compare l'étoile à un chemin qui ne passe pas par elle, si bien qu'une erreur de l'étoile ne se retrouve pas des deux côtés. `assert_reconciliation_ventes`, la valeur du fait contre les en-têtes de l'ERP ; `assert_production_contre_compteur`, les véhicules de `fct_production` contre le compteur des convoyeurs ; `assert_refus_hors_garantie`, les refus contre la règle recalculée depuis les dates de garantie ; `assert_jalons_dans_l_ordre`, l'ordre des jalons d'une réclamation ; `assert_compteur_heures_moteur`, la hausse du compteur contre la durée des sorties. *Théorie : `Theorie_09_AE.md`, section 5.*

20. Ce que les témoins ne voient pas. L'attribution d'une vente à un palier : 94 628 792,88 $ déplacés par une histoire perdue, sans qu'un témoin bronche, puisque tous comparent des totaux ou des règles où le palier n'entre pas. La preuve par les lignes le voit, à condition d'avoir mis l'entrepôt d'avant de côté : `dim_concessionnaire` et `fct_ventes` y sortiraient en écart. *Théorie : `Theorie_09_AE.md`, section 5.*

21. L'après-vente. Les semaines d'assemblage du 20 et du 27 octobre 2025 : 16 réclamations de refroidissement sur 119 motoneiges vendues, et 18 sur 118. Les dix-neuf autres semaines, d'août à décembre, en portent de zéro à cinq chacune, 24 en tout. *Théorie : `Theorie_09_AE.md`, section 6.*

22. L'usine. Par le 95e centile de la vibration moyenne en marche du compresseur 2, jour par jour, sur tout l'instantané : 3,28 millimètres par seconde, pour une médiane de 2,47 sur 120 jours. Six jours en sortent, du 21 au 28 octobre 2025. *Théorie : `Theorie_09_AE.md`, section 6.*

23. Joindre les deux bouts. La date d'une heure de l'usine convertie vers l'heure de Boréal au Cours 06, sans quoi une heure sur cinq change de jour ; la clé du véhicule recalculée par le même hachage dans cinq faits ; et l'actif, ligne d'assemblage comme compresseur, en dimension conforme. Aucun n'a été écrit pour l'incident : c'est la matrice de bus du Cours 02. *Théorie : `Theorie_09_AE.md`, section 6.*

24. Le `README.md`. Il dit ce qu'est le dépôt, ce qu'il livre, ce qu'il suppose avant de commencer, les commandes dans l'ordre avec ce qu'elles doivent afficher, et ce qui n'est pas dans le dépôt, avec la raison. Il ne raconte pas le code, que les en-têtes expliquent, ni le détail des produits, qui vit dans `LIVRAISON.md`. L'épreuve : quelqu'un qui ne vous connaît pas refait le projet en le suivant, sans vous appeler. *Théorie : `Theorie_09_AE.md`, section 7.*

25. L'étiquette. Un nom posé sur un commit, avec un auteur, une date et un message, comme un commit lui-même : `git tag -a v1.0 -m "..."`. On la pose pour que d'autres citent un état qui ne bougera plus quand `main` avancera. Elle ne part pas avec `git push` : `git push origin v1.0`. La `v2.0` viendrait d'un changement qui casse un consommateur, une colonne de `fct_ventes` renommée par exemple, que le contrat refuserait et que l'`exposure` dit à qui annoncer. *Théorie : `Theorie_09_AE.md`, section 7.*

© 2026 iAlexMG. Tous droits réservés.

Questions pratiques

P1. Deux environnements, une comparaison.

```
dbt build --exclude snp_concessionnaire
```

```
select
    (select count(*) from (
        select * from marts.fct_ventes except all select * from dev_marts.fct_ventes
    )) as manquent,
    (select count(*) from (
        select * from dev_marts.fct_ventes except all select * from marts.fct_ventes
    )) as en_trop;
```

La construction sur `dev` rend `PASS=128`, et la comparaison 0 et 0, pour 80 618 lignes de chaque côté. Pas de fichier à attacher : les deux environnements vivent dans le même entrepôt, à des schémas différents, par la macro du Cours 07. La dimension des concessionnaires de `dev` lit le snapshot de `snapshots`, partagé entre les cibles, et porte donc les mêmes 205 versions que la production. Puis `dbt run-operation nettoyer_dev` jette les trois schémas de travail. *Théorie : `Theorie_09_AE.md`, section 5.*

P2. La garde qui regarde la livraison. Mesuré sur une copie sans `argent` : la garde passe, les cinq passes du snapshot tournent, et la construction échoue au bout d'une minute, `PASS=120 ERROR=2 SKIP=6`, sur `fct_disponibilite_actifs` et `assert_production_contre_compteur`, avec `Catalog Error: Table with name "argent.usine_horaire" does not exist because schema "argent" does not exist.` Le défaut était visible avant la première écriture. Le bloc à ajouter en tête de la macro, avant le compte des schémas du projet :

```
{% set requete_recu %}
    select table_schema, count(*) as tables
    from information_schema.tables
    where table_schema in ('raw', 'argent')
    group by table_schema
{% endset %}
{% set recu = {} %}
{% for ligne in run_query(requete_recu).rows %}
    {% do recu.update({ligne[0]: ligne[1]}) %}
{% endfor %}
{% if recu.get('raw', 0) < 22 or recu.get('argent', 0) < 3 %}
    {{ exceptions.raise_compiler_error(
        "exiger_entrepot_neuf : la livraison de Data Engineering est incomplete, raw "
        ~ recu.get('raw', 0) ~ " tables sur 22, argent " ~ recu.get('argent', 0) ~ " sur 3."
    ) }}
{% endif %}
```


© 2026 iAlexMG. Tous droits réservés.

Sur la copie : `la livraison de Data Engineering est incomplete, raw 22 tables sur 22, argent 0 sur 3.`, code 1, en huit secondes au lieu d'une minute. Sur un entrepôt complet et neuf, la garde passe comme avant. Le compte vérifie un nombre de tables, pas leur contenu : c'est la limite, et le kit 02 ou `LIVRAISON.md` de Data Engineering disent le reste. *Théorie : `Theorie_09_AE.md`, section 4.*

P3. Le rapport d'exécution.

```
import collections
import json

r = json.load(open("target/run_results.json", encoding="utf-8"))
compte = collections.Counter(
    (x["unique_id"].split(".")[0], x["status"]) for x in r["results"]
)
for (genre, statut), n in sorted(compte.items()):
    print(f"{genre:10} {statut:8} {n:>4}")
print(len(r["results"]), "resultats en", round(r["elapsed_time"], 1), "s")
lents = sorted(r["results"], key=lambda x: x["execution_time"], reverse=True)[:5]
for x in lents:
    print(f"  {x['execution_time']:6.2f} s  {x['unique_id']}")
```

Stable d'une exécution à l'autre : 129 résultats, 28 modèles en `success`, 1 seed, 98 tests et le test unitaire en `pass`, l'`exposure` en `no-op`. Les trois modèles `ephemeral` n'y sont pas, n'ayant rien à exécuter. Instable : les durées, 4,8 puis 4,6 secondes en tout, et le palmarès, `fct_sorties` en tête les deux fois, puis `fct_codes_sortie` ou `fct_reclamations` selon l'exécution. Une durée d'un nœud se compare sur plusieurs exécutions, jamais sur une seule. *Théorie : `Theorie_09_AE.md`, section 4.*

P4. L'incident en deux lignes.

```
with usine as (
    select f.cle_date as jour,
           avg(f.vibration_moyenne) filter (where f.part_en_marche > 0.5) as vibration
    from marts.fct_disponibilite_actifs as f
    inner join marts.dim_actif as a on a.cle_actif = f.cle_actif
    where a.actif_id = 'UTL-CMP2'
    group by 1
),

ligne as (
    select p.cle_date_assemblage as jour, count(*) as assemblees,
           count(*) filter (where p.cle_vehicule in (
               select r.cle_vehicule from marts.fct_reclamations as r
               where r.cause = 'REFROIDISSEMENT'
           )) as refroidissement
```


© 2026 iAlexMG. Tous droits réservés.

```
    from marts.fct_production as p
    inner join marts.dim_actif as a on a.cle_actif = p.cle_actif
    where a.actif_id = 'LMN'
    group by 1
)

select
    case when u.vibration > (select quantile_cont(vibration, 0.95) from usine)
        then 'au-dessus' else 'dans la bande' end as vibration,
    count(*) as jours,
    sum(l.assemblees) as assemblees,
    sum(l.refroidissement) as refroidissement,
    round(100.0 * sum(l.refroidissement) / sum(l.assemblees), 2) as taux
from usine as u
inner join ligne as l on l.jour = u.jour
where u.vibration is not null
group by 1 order by 1;
```

Vibration	Jours	Assemblées	Refroidissement	Taux
au-dessus	6	144	31	21,53 %
dans la bande	105	1 823	16	0,88 %

Vingt-quatre fois plus. Les jours où le compresseur ne tournait pas n'ont pas de vibration et sortent du calcul, le 29 octobre compris ; c'est un choix à dire, pas un détail. *Théorie : `Theorie_09_AE.md`, section 6.*

P5. La liste du rappel.

```
create or replace temp view exposees as
select p.cle_vehicule
from marts.fct_production as p
inner join marts.dim_actif as a on a.cle_actif = p.cle_actif
where a.actif_id = 'LMN'
  and p.cle_date_assemblage between date '2025-10-15' and date '2025-10-28';

select d.province, count(*) as a_rappeler
from marts.fct_enregistrements as e
inner join marts.dim_concessionnaire as d on d.cle_concessionnaire = e.cle_concessionnaire
where e.cle_vehicule in (select cle_vehicule from exposees)
  and e.cle_vehicule not in (
      select r.cle_vehicule from marts.fct_reclamations as r where r.cause = 'REFROIDISSEMENT'
```


© 2026 iAlexMG. Tous droits réservés.

```
  )
group by 1 order by 2 desc;
```

203 motoneiges, les 237 vendues moins les 34 déjà réclamées, chez 20 concessionnaires : 64 en Ontario, 52 au Québec, 37 en Colombie-Britannique, 20 en Nouvelle-Écosse, 10 au Nouveau-Brunswick, 9 au Manitoba, 7 en Alberta et 4 en Saskatchewan. La province vient de la version d'époque de la dimension, mais elle n'est pas historisée : c'est celle d'aujourd'hui, ce qui est justement ce qu'il faut pour savoir qui appeler. *Théorie : `Theorie_09_AE.md`, section 6.*

P6. La documentation sous surveillance.

```
import json
import sys

m = json.load(open("target/manifest.json", encoding="utf-8"))
manquent = [
    n["name"] for n in m["nodes"].values()
    if n["resource_type"] in ("model", "seed", "snapshot") and not n["description"].strip()
]
manquent += [s["name"] for s in m["sources"].values() if not s["description"].strip()]
for nom in manquent:
    print("sans description :", nom)
print(len(manquent), "objet(s) sans description")
sys.exit(1 if manquent else 0)
```

```
      - name: La documentation, sans trou
        run: python outils/verifier_descriptions.py
```

L'étape se place après « Le fond, par dbt build », qui vient d'écrire `target/manifest.json`, et avant la génération de la documentation : une documentation publiée avec un trou n'a plus besoin d'être publiée. Vérifiée dans les deux sens, sur une répétition locale de la CI : `0 objet(s) sans description` et le code 0 sur le projet du labo ; la description de `fct_ventes` retirée, `sans description : fct_ventes`, et le code 1. Avant l'étape 2 du labo, le même script aurait nommé 18 objets, et aucune source. *Théorie : `Theorie_09_AE.md`, section 7.*

P7. Un test pour l'histoire. Dans `tests/assert_histoire_des_paliers.sql` :

```
with versions as (

    select
        code_concessionnaire,
        dbt_valid_from,
        dbt_valid_to,
        lead(dbt_valid_from) over (
            partition by code_concessionnaire
            order by dbt_valid_from, dbt_valid_to
        ) as debut_suivante
```


© 2026 iAlexMG. Tous droits réservés.

```
    from {{ ref('snp_concessionnaire') }}

)

select
    code_concessionnaire,
    dbt_valid_from,
    dbt_valid_to,
    'fin avant debut' as defaut
from versions
where dbt_valid_to <= dbt_valid_from

union all

select
    code_concessionnaire,
    dbt_valid_from,
    dbt_valid_to,
    'chevauchement' as defaut
from versions
where debut_suivante < coalesce(dbt_valid_to, date '9999-12-31')
```

Au vert sur l'entrepôt refait. Sur la copie reconstruite par-dessus : `FAIL 98`, 49 versions qui finissent avant de commencer et 49 chevauchements, un de chaque par concessionnaire touché. La première écriture de ce test joignait le snapshot à lui-même en excluant la même version par son `dbt_scd_id`, et ne voyait que les 49 inversions : les versions en double portent le même identifiant, puisque dbt le calcule sur la clé et la date. Comparer chaque version à la suivante, par `lead()`, ne dépend pas de cet identifiant. Le test passe aussi le linteur, et sur la structure vide de la CI. *Théorie : `Theorie_09_AE.md`, section 4.*

P8. Le troisième bout.

```
with premier_code as (
    select c.cle_vehicule, min(c.cle_date) as premier_code
    from marts.fct_codes_sortie as c
    where c.code_anomalie in ('T310', 'T320')
      and c.cle_vehicule in (select cle_vehicule from exposees)
    group by 1
),

premiere_reclamation as (
    select r.cle_vehicule, min(r.cle_date_ouverture) as premiere_reclamation
    from marts.fct_reclamations as r
    where r.cause = 'REFROIDISSEMENT'
```


© 2026 iAlexMG. Tous droits réservés.

```
      and r.cle_vehicule in (select cle_vehicule from exposees)
    group by 1
)

select count(*) as les_deux,
       min(premier_code) as plus_tot_code,
       min(premiere_reclamation) as plus_tot_reclamation,
       count(*) filter (where premier_code < premiere_reclamation) as code_avant,
       median(premiere_reclamation - premier_code) as ecart_median_jours
from premier_code
inner join premiere_reclamation using (cle_vehicule);
```

La vue `exposees` est celle de P5. 95 motoneiges de la cohorte sont connectées ; 17 ont levé un code du circuit de refroidissement ; 12 ont ensuite une réclamation de refroidissement, et chez les 12 le code vient avant, de 12 jours en médiane, le premier le 2025-12-18 contre une première réclamation le 2025-12-27. Restent 5 véhicules qui ont levé un code sans réclamer : ce sont les premiers à appeler, avant qu'ils tombent en panne chez leur client. *Théorie : `Theorie_09_AE.md`, section 6.*

Diagnostics

E1. Au vert, dans le vide. Sans `--target prod`, la cible par défaut est `dev`, et la macro du Cours 07 préfixe les schémas : tout a atterri dans `dev_staging`, `dev_marts` et `dev_presentation`, au vert. Le snapshot seul est au bon endroit, parce que son schéma est fixé par `target_schema` et ne passe pas par la macro. La comparaison regarde les schémas de production : 29 relations n'existent que dans l'entrepôt d'avant, et une seule, le snapshot, se compare. La garde n'a rien empêché, puisqu'elle cherchait les schémas `dev_*`, vides. C'est l'argument du script : `--target prod` y est écrit huit fois, une fois pour toutes. *Théorie : `Theorie_09_AE.md`, section 4.*

E2. Le changement oublié. L'entrepôt d'avant a été construit par un code qui n'est pas dans le dépôt : les libellés corrigés de Terre-Neuve-et-Labrador et de l'Île-du-Prince-Édouard sont passés dans `stg_regions`, puis dans les 1 326 clients et les 13 versions de concessionnaires de ces provinces. La copie neuve n'a vu que le code commité, et la comparaison le dit, alors que les 128 tests de l'entrepôt d'avant étaient au vert. Aucun des deux n'a « raison » d'office : le dépôt fait foi, et la correction contrevient à la règle du Cours 04, le `STAGING` ne réécrit pas la source. On la retire, ou, si l'équipe la veut, on la met ailleurs, commitée et relue. C'est exactement ce que la reconstruction depuis une copie sert à trouver. *Théorie : `Theorie_09_AE.md`, section 1.*

E3. Toutes les vues différentes. Chaque vue porte dans sa définition le nom de la base qu'elle lit, et la base porte le nom du fichier : refaites dans `boreal_neuf.duckdb`, les quatorze vues lisent `boreal_neuf.staging...` au lieu de `boreal.staging...`. Les tables, elles, n'en portent pas, et sont égales. Il doit refaire dans un fichier nommé `boreal.duckdb`, ailleurs ou après avoir mis l'ancien de côté sous un autre nom, comme le labo le fait. Un nom de fichier fait partie de ce que le projet suppose, et le `README.md` le dit. *Théorie : `Theorie_09_AE.md`, section 5.*

E4. La preuve qui se compare à elle-même. La vue de l'entrepôt d'avant, attachée à côté du refait, lit `boreal.marts.fct_ventes`, donc les tables du refait : les deux côtés de la comparaison calculent la même chose, et rendent 0 quoi qu'il arrive à l'ancien fait. Pour les tables, sa preuve vaut celle du labo ; pour les

© 2026 iAlexMG. Tous droits réservés.

quatorze vues, elle ne vaut rien, et c'est pire que rien, parce qu'elle affiche un zéro rassurant. Une vue se compare par sa définition, et son contenu suit des tables, comparées elles. *Théorie : `Theorie_09_AE.md`, section 5.*

E5. Les lots dans le désordre. Le lot de juillet 2024 présenté avant celui de juin fait fermer par le lot de juin des versions ouvertes en juillet : 302 versions, dont 49 qui finissent avant de commencer, et des chevauchements chez les mêmes 49 concessionnaires. La dimension passe ses tests, parce qu'ils vérifient l'unicité d'un code à une date de début, et que les versions fautives ont des débuts différents. Les faits tombent, parce que leur jointure temporelle trouve deux versions valides pour une même vente et double la ligne : 645 ventes, 164 enregistrements, 43 réclamations, et la réconciliation. Le test de P7 l'aurait vu à la source : `FAIL 146`, 49 inversions et 97 chevauchements, avant qu'un seul fait ne se construise. *Théorie : `Theorie_09_AE.md`, section 4.*

Questions à développer

A. Refaire ou restaurer. La première équipe peut revenir à hier en une heure, et elle garde tout ce que ses sources ont oublié, l'histoire de ses snapshots comprise. Mais elle ne sait pas si son projet se refait : le jour où la sauvegarde est corrompue ou trop vieille, elle découvre un geste manuel oublié, une table orpheline qui nourrissait un rapport, un changement jamais commité, trois défauts que le capstone a trouvés en effaçant. La seconde sait que son code suffit, et elle le prouve chaque dimanche. Mais tout ce que ses sources ne gardent pas disparaît à la première reconstruction : mesuré sur Boréal, une histoire perdue déplace 94 628 792,88 $ de palier sans qu'un test tombe. Pour Boréal, les deux : refaire régulièrement, avec la garde et les trois preuves, pour tout ce qui se dérive ; sauvegarder le schéma `snapshots`, seule partie qui accumule, et le déclarer permanent sur une plateforme qui le rendrait *transient*. La reconstruction prouve le code, la sauvegarde protège l'histoire, et aucune ne remplace l'autre. *Théorie : `Theorie_09_AE.md`, section 3.*

B. Fini. La phrase confond « rien n'échoue » et « tout est juste ». Le capstone a mesuré trois cas où tous les tests passent et où le projet ne l'est pas : une reconstruction sans les extraits gardés, `PASS=128` et 17 733 ventes au mauvais palier ; un entrepôt bâti par un changement jamais commité, 128 tests au vert et 2 679 lignes que le dépôt ne sait plus produire ; une reconstruction sans `--target prod`, au vert dans des schémas que personne ne lit. Les tests vérifient ce qu'on leur a demandé ; ils ne disent ni que le projet se refait, ni que ce qu'il refait est ce qui était là, ni qu'une autre personne saurait le reprendre. Fini, c'est les trois promesses de la section 1, et chacune a sa preuve. *Théorie : `Theorie_09_AE.md`, sections 1 et 5.*

C. Une CI qui refait tout. Pour : c'est la seule CI qui verrait un défaut de données avant la fusion, un filtre retiré qui fait tomber la réconciliation, une attribution qui glisse, ce que la CI sur structure vide laisse passer (Cours 08). Contre, trois coûts. Le temps : 68 secondes pour Boréal, dont quarante de snapshots, contre moins d'une minute pour la CI actuelle, et la reconstruction grossit avec les lots et les snapshots. L'accès : il faut une copie des vraies données, donc un secret, un rôle en lecture, et sur une plateforme une facture à chaque demande de tirage. Et la comparaison demande un entrepôt d'avant, donc un état de production conservé. Le partage raisonnable garde la CI sur structure vide pour chaque demande de tirage, et fait la reconstruction complète avec ses trois preuves à un rythme fixe, et avant chaque version étiquetée, là où son coût se justifie. *Théorie : `Theorie_09_AE.md`, sections 4 et 5.*

Mises en situation

M1. Coopérative agricole de Roberval. J'accepte l'idée pour tout ce qui se dérive de la source : refaire chaque mois depuis le dépôt est la meilleure preuve qu'il suffit, avec une garde et une comparaison à l'état d'avant. Je refuse d'effacer le snapshot des catégories : le logiciel ne livre que l'état courant, donc une

© 2026 iAlexMG. Tous droits réservés.

reconstruction rendrait une seule version par membre, et chaque changement de catégorie passé serait attribué à la catégorie d'aujourd'hui, sans qu'un test le voie. Je mets en place une reconstruction qui exclut le schéma des snapshots, une sauvegarde de ce schéma avant chaque passe et une table permanente sur leur plateforme, un test d'histoire comme celui de la question P7, et, à terme, une demande au fournisseur du logiciel : un extrait daté chaque mois, gardé, qui rendrait l'histoire refaisable. *Théorie : `Theorie_09_AE.md`, section 3.*

M2. Distributeur alimentaire de Rivière-du-Loup. D'abord les onze orphelines : les nommer, trouver qui les lit, les jeter ou les rattacher à un modèle, puis une garde qui refuse un entrepôt non vide, pour qu'elles ne reviennent pas. Ensuite le temps : si chaque passe coûte surtout le démarrage de dbt, comme sur Boréal, 30 snapshots rejoués sur 24 lots font 720 commandes, et c'est leur nombre qu'il faut réduire, pas les lignes. Rejouer seulement les lots où une valeur suivie change, mesurés d'avance, ou sauvegarder les snapshots et ne refaire que le reste. Ensuite les preuves, dans l'ordre : une comparaison ligne à ligne dans les deux sens, tables par leurs lignes et vues par leur définition ; les témoins indépendants qui existent, et un par étoile qui n'en a pas ; un test d'histoire sur chaque snapshot. Enfin, une reconstruction complète avant chaque version étiquetée, et une mesure du temps à chaque fois, pour voir le jour où il repart. *Théorie : `Theorie_09_AE.md`, sections 4 et 5.*

M3. Pêcheries de Gaspé. D'abord le `README.md` : ce qu'est le dépôt, ce qu'il suppose, les commandes dans l'ordre et ce qu'elles affichent, ce qui n'est pas dans le dépôt et où le trouver. Ensuite les descriptions : chaque modèle, chaque source, chaque snapshot, compté par un script dans la CI pour que le trou ne revienne pas. Ensuite le document de livraison, produit par produit, avec les comptes attendus et ce qu'un consommateur doit savoir. Enfin une étiquette sur l'état livré. L'épreuve se fait avant de partir, pas après : l'analyste efface une copie de l'entrepôt et refait le projet en suivant le `README.md`, seul, pendant que vous regardez sans parler. Chaque question qu'il pose est une ligne qui manque ; on l'écrit, et on recommence jusqu'à ce qu'il n'en pose plus. *Théorie : `Theorie_09_AE.md`, section 7.*

© 2026 iAlexMG. Tous droits réservés.